# Sierra 중간표현 핵심

## 개요

Sierra(Safe Intermediate Representation for Rust-like Assembly)는 Cairo 언어와 CASM(Cairo Assembly) 사이의 중간 표현입니다. Sierra는 "안전성"을 보장하면서도 효율적인 컴파일을 가능하게 하는 핵심적인 역할을 담당합니다.

## Sierra의 설계 철학과 목적

### 1. 안전성 보장 (Safety Guarantees)

Sierra의 가장 중요한 설계 목표는 **구조적 안전성**입니다:

```
//! Sierra is an intermediate representation between high level Cairo and compil

//! such as CASM. Sierra code is guaranteed to be "safe"* by construction.

//! Sierra has a primitive, yet rich typing system to express all high level cod

//! safety and allowing for efficient compilation down to the target.

//!

//! Safety - this means a few things:

//! 1. There are no "panics" / "runtime errors". Every function is guaranteed to

//! 2. There are no infinite loops. Moreover, every program "counts" its own ste

//! the limit is reached.

//! 3. Builtin library functions are always used correctly.
```

**안전성의 세 가지 핵심 원칙:**

1. **패닉 없음**: 모든 함수는 반드시 반환값을 가집니다
2. **무한 루프 없음**: 모든 프로그램은 자체적으로 실행 단계를 계산하고 제한에 도달하면 반환합니다
3. **올바른 빌트인 사용**: 빌트인 라이브러리 함수들이 항상 올바르게 사용됩니다

### 2. Cairo와 CASM 사이의 브릿지 역할

Sierra는 고수준 Cairo 코드와 저수준 CASM 코드 사이의 중간 계층으로서:

- **추상화 제공**: Cairo의 복잡한 언어 구조를 단순한 명령어로 변환
- **최적화 기회**: 타입 정보와 제어 흐름을 유지하면서 최적화 수행
- **안전성 검증**: 컴파일 시점에 안전성 속성 검증

## Sierra 프로그램 구조

### 1. 기본 프로그램 구조

```
/// A full Sierra program.
#[derive(Clone, Debug, Eq, PartialEq, Serialize, Deserialize)]
pub struct Program {
/// Declarations for all the used types.
pub type_declarations: Vec<TypeDeclaration>,
/// Declarations for all the used library functions.
pub libfunc_declarations: Vec<LibfuncDeclaration>,
/// The code of the program.
pub statements: Vec<Statement>,
```

```
/// Descriptions of the functions - signatures and entry points.
pub  funcs: Vec<Function>,
}
```

Sierra 프로그램은 네 가지 주요 구성요소로 이루어집니다:

1. **타입 선언** ( `type_declarations` ): 사용되는 모든 구체적 타입들의 선언
2. **라이브러리 함수 선언** ( `libfunc_declarations` ): 사용되는 모든 라이브러리 함수들의 선언
3. **명령문** ( `statements` ): 실제 프로그램 코드
4. **함수** ( `funcs` ): 함수 시그니처와 진입점 정보

## 2. 실제 Sierra 코드 예제

```
type felt252 = felt252;
type GasBuiltin = GasBuiltin;
type RangeCheck = RangeCheck;
type NonZeroInt = NonZero<felt252>;

libfunc branch_align = branch_align;
libfunc felt252_add = felt252_add;
libfunc felt252_const_0 = felt252_const<0>;
libfunc felt252_is_zero = felt252_is_zero;
libfunc withdraw_gas = withdraw_gas;

// Statement #1 - tests if n == 0.
```

```
felt252_is_zero(n) { fallthrough() LoopSetup(n) };

branch_align() -> ();

store_temp_rc(rc) -> (rc);

redeposit_gas(gb) -> (gb);

return(rc, gb, one);



Fibonacci@0(rc: RangeCheck, gb: GasBuiltin, n: felt252) -> (RangeCheck, GasBuilt
```

## Sierra 명령어 세트 상세 분석

### 1. 명령문 구조

```rust
/// A possible statement.

#[derive(Clone, Debug, Eq, PartialEq, Serialize, Deserialize)]

pub  enum GenStatement<StatementId> {

Invocation(GenInvocation<StatementId>),

Return(Vec<VarId>),

}
```

Sierra 명령문은 두 가지 형태입니다:

- **함수 호출** ( `Invocation` ): 라이브러리 함수 호출
- **반환** ( `Return` ): 함수에서 값 반환

### 2. 함수 호출 구조

```
/// An invocation statement.
#[derive(Clone, Debug, Eq, PartialEq, Serialize, Deserialize)]
pub struct GenInvocation<StatementId> {
/// The called libfunc.
pub libfunc_id: ConcreteLibfuncId,
/// The arguments consumed by the libfunc's invocation.
pub args: Vec<VarId>,
/// The possible branches to continue to after the invocation.
pub branches: Vec<GenBranchInfo<StatementId>>,
}
```

각 함수 호출은:

- **함수 ID**: 호출할 구체적 라이브러리 함수
- **인수**: 함수에 전달할 변수들
- **분기**: 함수 실행 후 가능한 제어 흐름 경로들

## 3. 핵심 라이브러리 함수 카테고리

Sierra의 라이브러리 함수들은 계층적으로 구성됩니다:

```
define_libfunc_hierarchy! {
pub enum CoreLibfunc {
// 메모리 관리
ApTracking(ApTrackingLibfunc),
Drop(DropLibfunc),
```

```
Dup(DupLibfunc),

// 기본 타입 연산

Felt252(Felt252Libfunc),

Bool(BoolLibfunc),

// 제어 흐름

BranchAlign(BranchAlignLibfunc),

UnconditionalJump(UnconditionalJumpLibfunc),

// 함수 호출

FunctionCall(FunctionCallLibfunc),

// 가스 관리

Gas(GasLibfunc),

// 데이터 구조

Array(ArrayLibfunc),

Struct(StructLibfunc),

Enum(EnumLibfunc),

// 암호화 연산

Pedersen(PedersenLibfunc),

Poseidon(PoseidonLibfunc),

// Starknet 특화

Starknet(StarknetLibfunc),

}, CoreConcreteLibfunc

}
```

**주요 함수 카테고리별 설명:**

**메모리 관리**:

- `store_temp<T>` : 임시 메모리에 값 저장
- `store_local<T>` : 로컬 메모리에 값 저장
- `drop<T>` : 값 해제
- `dup<T>` : 값 복제

**기본 연산**:

- `felt252_add` : felt252 덧셈
- `felt252_sub` : felt252 뺄셈
- `felt252_is_zero` : 0인지 검사

**제어 흐름**:

- `branch_align` : 분기 정렬
- `jump` : 무조건 점프
- `function_call<F>` : 함수 호출

# Sierra 타입 시스템

## 1. 타입 선언 구조

```
/// Declaration of a concrete type.
#[derive(Clone, Debug, Eq, PartialEq, Serialize, Deserialize)]
pub struct TypeDeclaration {
/// The id of the declared concrete type.
pub id: ConcreteTypeId,
```

```
pub long_id: ConcreteTypeLongId,
pub declared_type_info: Option<DeclaredTypeInfo>,
}
```

## 2. 타입 정보

```
/// Declaration of a concrete type info.
#[derive(Clone, Debug, Eq, PartialEq, Hash, Serialize, Deserialize)]
pub struct DeclaredTypeInfo {
/// Can the type be stored by any of the store commands.
pub storable: bool,
/// Can the type be (trivially) dropped.
pub droppable: bool,
/// Can the type be (trivially) duplicated.
pub duplicatable: bool,
/// The size of an element of this type.
pub zero_sized: bool,
}
```

각 타입은 네 가지 속성을 가집니다:

- **저장 가능성** ( `storable` ): store 명령어로 저장 가능한지
- **해제 가능성** ( `droppable` ): 자동으로 해제 가능한지
- **복제 가능성** ( `duplicatable` ): 복제 가능한지

- **크기** ( `zero_sized` ): 크기가 0인지

## 3. 핵심 타입들

```
define_type_hierarchy! {
pub  enum CoreType {
// 기본 타입
Felt252(Felt252Type),
// 정수 타입
Uint8(Uint8Type),
Uint16(Uint16Type),
Uint32(Uint32Type),
Uint64(Uint64Type),
Uint128(Uint128Type),
// 데이터 구조
Array(ArrayType),
Struct(StructType),
Enum(EnumType),
// 특수 타입
Box(BoxType),
Nullable(NullableType),
NonZero(NonZeroType),
// 빌트인
GasBuiltin(GasBuiltinType),
```

```
RangeCheck(RangeCheckType),

// Starknet

Starknet(StarknetType),

}, CoreTypeConcrete

}
```

## Sierra 메모리 모델

### 1. 메모리 참조 정보

```
/// Information regarding the reference created as an output of a library functi

#[derive(Debug, Clone)]

pub  enum OutputVarReferenceInfo {

/// The output value is exactly the same as one of the parameters.

SameAsParam { param_idx: usize },

/// The output value is a part of one of the parameters.

PartialParam { param_idx: usize },

/// The output was allocated as a temporary variable and it is at the top of the

NewTempVar { idx: usize },

/// The output was allocated as a local variable.

NewLocalVar,

/// The output is the result of a computation.

Deferred(DeferredOutputKind),
```

```
/// All the output cells are of the form `[ap/fp + const]`.

SimpleDerefs,

/// The output is a of size 0.

ZeroSized,

}
```

## 2. 메모리 할당 전략

Sierra는 세 가지 주요 메모리 영역을 사용합니다:

**임시 변수 (Temporary Variables)**:

- 스택 최상단에 연속적으로 할당
- 함수 호출 간에 전달되는 값들
- `store_temp<T>` 명령어로 생성

**로컬 변수 (Local Variables)**:

- 함수 프레임 내에서 할당
- `alloc_local<T>` 로 할당, `store_local<T>` 로 초기화
- 함수 종료 시 자동 해제

**매개변수 참조**:

- 기존 매개변수를 그대로 사용하거나 부분적으로 사용
- 메모리 복사 없이 효율적인 참조

## 3. AP (Allocation Pointer) 추적

```
/// Describes the effect on the `ap` register in a given libfunc branch.
```

```rust
#[derive(Clone, Debug, Eq, PartialEq)]
pub enum SierraApChange {
/// The libfunc changes `ap` in an unknown way.
Unknown,
/// The libfunc changes `ap` in a known (during compilation) way.
Known { new_vars_only: bool },
/// The lib func is `branch_align`.
BranchAlign,
/// This is a function call, and the ap change should be fetched elsewhere.
FunctionCall(FunctionId),
}
```

AP 변화 추적은 메모리 안전성과 최적화에 중요합니다:

- **알려진 변화**: 컴파일 시점에 정확한 AP 변화량 계산 가능
- **알 수 없는 변화**: 런타임에 결정되는 변화
- **분기 정렬**: 분기 후 AP 정렬 필요
- **함수 호출**: 다른 함수의 AP 변화 정보 참조

## Cairo에서 Sierra로의 변환 과정

### 1. 변환 파이프라인

```rust
// Cairo HIR → Sierra 변환 과정
pub fn get_sierra_program_for_functions<'db>(
```

```
db: &'db dyn SierraGenGroup,
requested_function_ids: Vec<ConcreteFunctionWithBodyId<'db>>,
) -> Maybe<Arc<SierraProgramWithDebug<'db>>> {
// 1. 함수들을 pre-Sierra로 변환
let mut functions: Vec<Arc<pre_sierra::Function<'_>>> = vec![];
let mut statements: Vec<pre_sierra::StatementWithLocation<'_>> = vec![];
// 2. 의존성 함수들 수집
// 3. 라벨 해결 및 최종 Sierra 프로그램 생성
let (program, statements_locations) = assemble_program(db, functions, statements
}
```

## 2. 타입 변환

Cairo의 타입들이 Sierra 타입으로 변환됩니다:

```
// Cairo felt252 → Sierra felt252
type felt252 = felt252;

// Cairo Array<T> → Sierra Array<T>
type Array<felt252> = Array<felt252>;

// Cairo struct → Sierra struct
type Point = Struct<felt252, felt252>;
```

### 3. 함수 변환

Cairo 함수는 Sierra 함수로 변환되며, 각 함수는:

- 매개변수 타입 정보
- 반환 타입 정보
- 진입점 (entry point)
- 함수 본문 (Sierra 명령문들)

## Sierra와 CASM 사이의 관계

### 1. Sierra의 추상화 수준

Sierra는 CASM보다 높은 수준의 추상화를 제공합니다:

**Sierra 수준**:

```
felt252_add(a, b) -> (result);
```

**CASM 수준**:

```
[ap] = [fp - 3] + [fp - 4]; ap++
```

### 2. 안전성 보장의 전달

Sierra의 안전성 속성들이 CASM 생성 시 유지됩니다:

- **타입 안전성**: Sierra 타입 정보가 CASM 메모리 레이아웃 결정
- **제어 흐름 안전성**: Sierra 분기 구조가 CASM 점프로 안전하게 변환
- **메모리 안전성**: Sierra 메모리 모델이 CASM 메모리 접근 패턴 결정

### 3. 최적화 기회

Sierra는 CASM 생성 시 다양한 최적화를 가능하게 합니다:

- **레지스터 할당**: Sierra 변수 생명주기 정보 활용
- **메모리 레이아웃**: Sierra 타입 정보로 효율적인 메모리 배치
- **가스 최적화**: Sierra 가스 정보로 최적의 가스 사용량 계산

## 결론

Sierra 중간표현은 Cairo 컴파일러의 핵심 구성요소로서:

1. **안전성 보장**: 구조적으로 안전한 코드 생성 보장
2. **효율적인 변환**: Cairo와 CASM 사이의 효율적인 브릿지 역할
3. **최적화 기회**: 타입 정보와 제어 흐름 정보를 활용한 최적화
4. **확장성**: 새로운 타입과 연산의 쉬운 추가

Sierra의 설계는 Cairo 언어의 안전성과 성능을 동시에 달성하는 핵심적인 역할을 수행하며, provable computation의 요구사항을 만족시키는 중요한 중간 계층입니다.